SENDAZA LIQUIDITY ENGINE

Sprint 4: Executable Quote Engine

How a safe reference rate becomes an immutable customer offer

Delivered 2 September 2026

> **Final status:** Delivered. SLE can create an exact, short-lived BUY quote from a configured market, active quote policy, and fresh Sprint 3 reference snapshot. All nine migrations are applied to the isolated test database and Railway production. No production quote policy or pricing route was activated.

Connection to Earlier Sprints

Sprint 1 supplied the configurable asset, fiat, network, and market registry. Sprint 2 supplied authenticated private requests, correlation IDs, audit records, and reusable idempotency. Sprint 3 supplied fresh, reproducible reference-rate snapshots. Sprint 4 combines those foundations with versioned economics and freezes the result as a quote. Sprint 6 will later consume an unexpired quote to reserve inventory.

What Was Built

- `quote.ts` performs exact bigint fee, spread, and destination calculations.
- `quote-service.ts` selects server-controlled evidence, enforces expiry, persists the quote, and returns decimal strings.
- `prisma-quote.repository.ts` selects enabled configuration and stores every evidence field.
- `quote.controller.ts` exposes private `POST /api/v1/quotes` with authentication, audit, correlation, and idempotency infrastructure.
- The quote migrations add versioned policies, immutable quotes, active-policy uniqueness, and database checks that recompute fees, destination quantity, and disclosed spread.

End-to-End Example

```
Sendaza: BUY marketId + total debit "200000.00"
                    |
                    v
  enabled market + active policy + fresh Sprint 3 snapshot
                    |
                    v
  fee ceiling -> trade amount -> spread-adjusted rate
                    |
                    v
  destination crypto floors to asset atomic units
                    |
                    v
  immutable quote + expiry -> decimal-string response
```

The total debit includes purchase fees. A quote does not lock a Sendaza balance and does not reserve SLE inventory. Sendaza may display it, but a later purchase workflow must recheck expiry and reserve inventory.

Rules Protected

- No JavaScript floating-point arithmetic is used for money or asset quantities.
- Customer principal with excess fiat precision is rejected, never silently rounded.
- Percentage fees round upward; destination asset quantity rounds downward.
- The client cannot select the provider, pricing route, policy, backing network, or rounding rule.
- Quote expiry is never later than its reference snapshot expiry.
- Stored quote economics cannot be updated or deleted, and PostgreSQL rejects inconsistent evidence.

Verification Evidence

- 36 unit suites passed with 203 tests.
- Four focused endpoint, contract, orchestration, and repository suites passed with 25 tests.
- The real PostgreSQL suite passed 6 tests against the isolated `sle_test` database.
- Strict TypeScript passed; API and worker production builds compiled successfully.
- Railway production reports 9 migrations and an up-to-date schema.

Limitations and Next Dependency

No production quote policy or pricing route is active, so deployment does not begin live quoting. Quote insertion and generic idempotency-response completion are separate transactions: a crash between them leaves a durable quote and an in-progress request for reconciliation, while still blocking duplicate use of the key. Automated recovery must be resolved before real funds. Sprint 5 next synchronizes Fireblocks MPC treasury wallets and asset-network inventory; Sprint 6 connects accepted quotes to atomic reservations.

Glossary

Total debit: all fiat the customer authorizes. **Trade amount:** total debit minus fees. **Basis point:** one hundredth of one percent. **Atomic unit:** smallest supported unit of a fiat currency or crypto asset. **Evidence:** the exact policy and rate records that reproduce a quote. **Executable quote:** immutable short-lived economics that a later purchase may consume.